

RATE-SENTINEL HIGH LEVEL DESIGN (HLD)

Version: 1.0

Date: 2026-07-16

Project: rate-sentinel

1. PURPOSE AND SCOPE

Rate-Sentinel is a centralized gateway-side control plane for API protection and shared trust capabilities.

It enforces rate limits, authenticates callers, supports OTP workflows, ensures idempotent payment processing, and publishes notification intents to Kafka.

This HLD covers the system context, major components, key data flows, deployment architecture, and cross-cutting concerns. Detailed class-level behavior is documented in the LLD.

2. GOALS

- Enforce configurable per-client and per-route rate limiting with low latency.
- Provide JWT-based authentication and role-protected APIs.
- Offer reusable OTP generation and verification with lockout policy.
- Prevent duplicate payment execution using idempotency keys.
- Decouple notification delivery via asynchronous Kafka event publishing.
- Expose operational metrics and health probes for platform observability.

3. SYSTEM CONTEXT

External actors and dependencies:

- Client applications and internal services call REST APIs exposed by Rate-Sentinel.
- MySQL stores durable business and rule state.
- Redis stores fast-changing counters, lock keys, and cache entries.
- Kafka receives notification/payment events for downstream processing.
- Admin users manage rules through admin endpoints.
- Monitoring stack scrapes actuator metrics (Prometheus format).

4. HIGH-LEVEL ARCHITECTURE

Logical layers:

1) Edge and Security Layer

- Spring Security + JWT validation filter.
- Rate limiting filter intercepts calls before controller execution.

2) API and Orchestration Layer

- Controllers: Auth, OTP, Payment, Admin Rule Management.
- Services: RateLimitService, OTPService, PaymentService, NotificationDispatcherService.

3) Data and Messaging Layer

- MySQL via Spring Data JPA repositories.

- Redis for counters, lockout keys, and cache.
- Kafka topics for asynchronous event publication.

5. MAJOR BUILDING BLOCKS

- 'RateLimitFilter': computes caller identity, evaluates quota decision, returns 429 when required.
- 'RateLimitService': rule resolution and algorithm dispatch.
- 'RateLimitAlgorithmFactory': runtime strategy selection.
- 'SlidingWindowAlgorithm', 'TokenBucketAlgorithm', 'FixedWindowAlgorithm': Redis-backed controls.
- 'JWTAuthFilter' and 'JWTTokenProvider': token validation and identity propagation.
- 'OTPService': OTP generation/verification and lockout handling.
- 'PaymentService': idempotency guard (Redis + MySQL) and payment lifecycle transitions.
- 'NotificationDispatcherService': non-blocking event publishing to Kafka topics.

6. KEY RUNTIME FLOWS

6.1 REQUEST RATE-LIMIT ENFORCEMENT

1. Request enters filter chain.
2. 'JWTAuthFilter' validates token (if present) and sets 'clientId' request attribute.
3. 'RateLimitFilter' resolves effective 'clientId' and route.
4. 'RateLimitService' resolves applicable rule (exact > wildcard > global > fallback).
5. Selected algorithm checks allowance in Redis.
6. Response headers are set ('X-RateLimit-*').
7. If denied, HTTP 429 is returned; otherwise request continues to controller.

6.2 OTP FLOW

1. Client calls generate endpoint.
2. Service verifies lock status in Redis.
3. OTP is generated, hashed, persisted, and dispatched as notification event.
4. Verify endpoint checks latest active OTP, expiry, attempts, and hash match.
5. On excessive failures, lock key is set in Redis.

6.3 PAYMENT FLOW

1. Client submits payment with 'Idempotency-Key'.
2. Redis short-circuit check runs first; MySQL check is fallback durability layer.
3. For new key, payment is created and processed.
4. Payment status transitions to SUCCESS/FAILED and is persisted.
5. Notification event is emitted asynchronously.

7. DATA ARCHITECTURE (HIGH LEVEL)

Primary durable entities:

- 'RateLimitRule': clientId/route pair, requestLimit, windowSeconds, algorithm, active state.
- 'OTPRecord': identifier, channel type, hashed OTP, expiry, attempts, used flag.
- 'Payment': idempotency key (unique), clientId, amount/currency, status, failure reason, timestamps.

Fast-state Redis keys:

- Sliding window zset keys: 'sw:{clientId}:{route}'.
- Token bucket keys: token and refill timestamps.
- Rule cache keys: 'rate-limit-rules::{clientId}:{route}'.
- OTP lock keys: 'otp:lock:{identifier}'.
- Payment idempotency keys: 'payment:idem:{idempotencyKey}'.

8. DEPLOYMENT VIEW

Containerized local topology from 'docker-compose.yml':

- Kafka broker (KRaft mode) and Kafka UI.
- Redis.
- MySQL expected as external dependency (configured in application properties through env vars).
- Spring Boot application process.

Ports (as configured):

- App: 8080 (default Spring Boot)
- Kafka broker: 9092/29092
- Kafka UI: 8081
- Redis: 6379

9. SECURITY AND COMPLIANCE

- Stateless JWT auth for protected routes.
- BCrypt for OTP storage; no plaintext OTP persistence.
- Role protection for admin routes.
- Idempotency pattern mitigates duplicate financial processing risk.
- Secrets are externalized through environment variables and expected vault integration in production.

10. OBSERVABILITY AND OPERABILITY

- Health endpoints and readiness checks via Spring Actuator.
- Prometheus metrics exported at '/actuator/prometheus'.
- Counters include rate-limit pass/throttle events and service-domain events.
- Response headers expose real-time quota context to clients.

11. RISKS AND MITIGATIONS

- Risk: Caller identity mismatch causes unexpected rule misses.
Mitigation: wildcard and pattern rules, plus debug logs for identity and route.
- Risk: Concurrency overshoot in quota checks.
Mitigation: atomic Redis Lua script in sliding-window implementation.
- Risk: Duplicate payment retries.
Mitigation: Redis + DB idempotency check with TTL and unique key constraint.

12. ASSUMPTIONS AND OUT OF SCOPE

- Outbound notification delivery is handled by downstream consumers, not this service.
- OAuth/OIDC, user lifecycle, and refund processing are not in current scope.
- Multi-region active-active deployment is out of scope for v1.